

Mini Language Project Documentation

Project: Mini Programming Language and Interpreter

Repository root: PPL-mini-lang-proj

Generated for architecture review and CLI usage documentation

Part 1: Language Interpreter Logic

1. Overview

The project implements a compact educational programming language with a classic three-stage execution pipeline:

- Lexing: source text is converted into tokens.
- Parsing: tokens are assembled into an abstract syntax tree (AST).
- Interpretation: the AST is executed with an environment model and runtime checks.

The implementation separates concerns cleanly into src/core modules and keeps the language entry points simple for command-line and web use.

2. Execution pipeline

The central flow is:

- Source text is read from a file or console.
- Lexer scans characters and emits Token objects.
- Parser consumes tokens and produces AST node objects.
- Interpreter executes the AST and evaluates expressions.
- Environment stores values, supports nested scopes, and resolves variables.

This pattern matches a standard interpreter architecture used in scripting languages and classroom compilers.

3. File-by-file execution engine

src/core/tokens.py

Purpose:

- Defines Token dataclasses and keyword/operator lookup tables.
- Maps single-character operators (+, -, *, /, =, (,), {, }, etc.) to token kinds.
- Maps multi-character operators (==, !=, >=, <=) to token kinds.

Key role:

- This is the language contract for the lexer. Without these mappings, the parser cannot reliably recognize syntax.

src/core/lexer.py

Purpose:

- Scans raw source as characters.
- Ignores whitespace, comments (# and //), and line breaks.
- Produces NUMBER, STRING, IDENTIFIER, and keyword tokens.
- Raises LexerError for illegal characters, invalid numbers, or unterminated strings.

Mechanism:

- The Lexer keeps a current index and line counter.
- It detects operators, strings, numbers, and identifiers.
- It converts keyword words like let, print, if, else, while, function, return, true, false, and, or, end to uppercase token kinds.

This stage is the lexical front end of the interpreter.

src/core/ast_nodes.py

Purpose:

- Declares the AST node types as data classes.
- Each node represents a syntactic construct: Literal, Variable, Binary, Unary, Logical, Call, VarDeclaration, Assignment, PrintStatement, Block, IfStatement, WhileStatement, FunctionStatement, ReturnStatement.

Why it matters:

- The parser builds an AST tree using these nodes.
- The interpreter then pattern-matches on these object types while executing the program.

src/core/parser.py

Purpose:

- Consumes tokens and constructs an AST according to the grammar.
- Implements recursive-descent parsing for expressions and statements.

Important behavior:

- parse() repeatedly reads statement() until EOF.
- statement() recognizes declarations, assignments, print, if, while, function, return, and expression statements.
- expression() delegates to or_expr(), and_expr(), equality(), comparison(), term(), factor(), unary(), call(), and primary().
- call() handles function-call syntax and argument lists.
- block_until() and block() allow parsing nested blocks and control-flow bodies.

This is the grammar-driven core of the language. It matches the EBNF rules seen in docs/GRAMMAR.md.

src/core/environment.py

Purpose:

- Implements runtime scope resolution.
- Each Environment has a parent reference, allowing lexical scoping and nested function closures.

Behavior:

- define() adds a variable to the current environment.
- get() walks outward to parent scopes.
- assign() updates the nearest existing binding or raises an error.

This provides the interpreter's variable visibility model and supports closures.

src/core/interpreter.py

Purpose:

- Executes AST nodes produced by the parser.
- Evaluates expressions and performs runtime semantic checks.

Core mechanics:

- run() executes a list of statements in order.
- execute() dispatches by node type.
- evaluate() recursively evaluates literals, variables, unary operations, logical expressions, binary expressions, and function calls.
- binary() handles +, -, *, /, comparisons, equality, and inequality with type checks.
- _bool() ensures conditions are boolean values.
- num() and nums() enforce numeric checks.

Function model:

- UserFunction is created for function definitions.
- Each function keeps its defining closure and parameter list.
- When called, a new child Environment is created, parameters are bound, and execution runs in that local scope.
- ReturnSignal is raised to exit a function and return a value.

This is the heart of the language runtime.

src/main.py

Purpose:

- Public execution entry point for running the language programmatically.
- Reads file input or standard input.
- Calls Lexer -> Parser -> Interpreter.
- Catches language errors and prints them.

Behavior:

- `execute_source(source)` runs the full pipeline and returns outputs.
- `main()` supports script execution and a simple REPL-like interface if no file is supplied.

This is the CLI-facing wrapper that turns the core engine into a usable language runner.

4. Supported semantics and language features

The language supports:

- Variable declarations: `let x = 5`
- Assignment: `x = x + 1`
- Numbers, strings, booleans
- Arithmetic and comparisons
- Logical operators: `and`, `or`, `!`
- If/else blocks
- While loops
- Function definitions and function calls
- Lexical closure behavior
- Runtime errors for undefined variables, invalid type usage, division by zero, invalid conditions, and invalid calls

The interpreter uses dynamic runtime typing rather than static type checking.

5. Grammar and syntax contract

The grammar file in `docs/GRAMMAR.md` defines the language in EBNF style. It explicitly describes:

- Program structure
- Statements and blocks
- Expression precedence and operators
- Function and parameter syntax
- Return handling

The parser in `src/core/parser.py` is a direct implementation of that grammar.

6. Architectural summary

Overall architecture:

- Lexer: tokenizes source text.
- Parser: produces AST.
- Environment: resolves scope and variables.
- Interpreter: executes AST and runtime semantics.
- Main entry: runs files or input streams.

This is a straightforward, classroom-appropriate interpreter architecture that is readable and modular.

Part 2: Environment & CLI Integration

1. Project environment and runtime requirements

The project is Python-based and is designed to work in a regular command-line environment.

Requirements file:

- `requirements.txt`
- It indicates no third-party packages are required.
- Python 3.10+ is recommended.

This keeps the project lightweight and suitable for a local interpreter session or simple Windows command-line wrapper.

2. Root-level project entry points and runnable scripts

`main.py`

Purpose:

- A top-level script that wraps the interpreter for direct command-line use.
- It is designed to be executed with Python and accepts a filename or inline code.

Runtime examples:

- python main.py examples/sample_program.txt
- python main.py -c "let x = 5; print(x);"

This script is one of the simplest command-line launching points for the language.

server.py

Purpose:

- Runs a local HTTP server for a browser-based frontend.
- Serves files from server/web and exposes a POST endpoint /api/run.
- Calls the same interpreter pipeline used in the CLI, enabling UI execution.

It is important because it gives the language a web interface as well as a terminal interface.

3. Command-line launch and usage scripts

cmd/mini.cmd

Purpose:

- Windows batch launcher.
- Checks for compiled executable outputs in dist before falling back to running main.py.

Behavior:

- Executes a built mini.exe if present.
- Otherwise calls Python on the root main.py script.

This is a standard Windows wrapper for shipping the language as a command-line app.

cmd/build_windows.cmd

Purpose:

- Launches the PowerShell build script for Windows packaging.
- Intended to produce compiled executables and build artifacts.

cmd/mini-shell.cmd

Purpose:

- Opens a dedicated command shell window and runs the mini language script from there.

cmd/run.bat

Purpose:

- Starts the local server process for browser-based use.
- It runs server.py and pauses for user observation.

These scripts are the practical CLI integration layer for Windows-based usage.

4. PowerShell integration

powershell/mini.ps1

Purpose:

- Windows PowerShell equivalent to mini.cmd.
- Uses the compiled dist executables if available; otherwise runs Python main.py.

This script is essential for PowerShell users and matches the same command-line experience as the batch files.

powershell/build_windows.ps1

Purpose:

- Build script for packaging or bundling the language for Windows.
- Supports creating windows-friendly executable bundles.

5. Packaging and build metadata

mini.spec

Purpose:

- PyInstaller specification file.
- Tells the packaging tool what to compile into a standalone Windows executable.

Key items:

- Name: mini
- Console mode: true
- Inputs include main.py and bundled runtime metadata.

This is how the project can be turned into an executable for easier command-line use.

build/

Purpose:

- Generated build artifacts from PyInstaller or compiled packaging steps.
- Contains the intermediate packaging output for the mini application.

These files are mostly build output rather than source logic and should not be edited manually in normal development.

6. Functional usage commands

The following are the main commands to make the project work as a language tool:

Windows terminal / command prompt:

- python main.py examples/sample_program.txt
- python -m unittest discover -s tests -v
- python server.py

Using the batch wrappers:

- cmd\mini.cmd examples/sample_program.txt
- cmd\run.bat

PowerShell:

- powershell\mini.ps1 examples/sample_program.txt

Browser/web access:

- Start server.py
- Open <http://127.0.0.1:8000>

This shows the project is designed to operate both as a CLI interpreter and as a browser-hosted environment.

7. Server and web integration

The folder server/web contains the frontend UI for the language.

server/web/index.html

Purpose:

- Browser interface with editor, output panel, example selector, and documentation sections.

server/web/styles.css

Purpose:

- Styling and theming for the editor and app layout.

server/web/app.js

Purpose:

- Browser-side logic for example loading, theme switching, and calling the backend interpreter.

server/web/README.md

Purpose:

- Short instructions for how the web UI is intended to run and how it relates to the Python backend.

This makes the language usable in a visual environment while still relying on the same interpreter core.

8. Test environment and validation

tests/test_basic.py

Purpose:

- Validates core behavior using unittest.
- Covers basic print, loops, if/else, functions, and type error handling.

This is the execution verification layer for the interpreter's main language features.

9. Documentation and usage guidance

docs/GRAMMAR.md

Purpose:

- Defines the language grammar in EBNF.
- This is essential because the parser is built to match it.

README.md

Purpose:

- High-level overview of the language, architecture, supported features, and execution instructions.

START_HERE.txt

Purpose:

- Quick-start instructions for running the server and launching the language.

These files define the user-facing entry points and documentation narrative.

End of document: What each folder is for and what the files themselves actually do

Root folder

- README.md: project overview, architecture summary, supported features, and usage instructions.
- START_HERE.txt: startup checklist and direct usage notes.
- requirements.txt: lists runtime requirements; no external dependencies are needed.
- server.py: local HTTP server to power the language web interface.
- main.py: top-level program entry and direct script runner.
- mini.spec: PyInstaller specification for creating a Windows executable.

src/

Purpose: source code for the language runtime.

- src/__init__.py: package marker for Python module import compatibility.
- src/main.py: language runner for file and CLI execution.
- src/core/lexer.py: converts text into tokens.
- src/core/parser.py: turns tokens into an AST.
- src/core/interpreter.py: executes the AST and performs evaluation.
- src/core/environment.py: stores variable bindings and scope structure.
- src/core/ast_nodes.py: AST node definitions.
- src/core/tokens.py: token kinds and keyword maps.

docs/

Purpose: language specification and grammar documentation.

- docs/GRAMMAR.md: EBNF grammar for the language syntax.

examples/

Purpose: example language programs demonstrating behavior.

- sample_program.txt: a sample program showing basic language use.
- concepts_demo.txt: demonstration of major PPL concepts such as recursion, closures, and variables.
- function_test.txt: function-call and return examples.
- scope_test.txt: variable scope and closure examples.
- error_test.txt: invalid inputs and language error cases.
- demo/hello.mini: small sample or demo script in .mini format.

tests/

Purpose: automated validation.

- tests/test_basic.py: fundamental interpreter tests for output correctness and runtime errors.

cmd/

Purpose: Windows command-line shortcuts and launch scripts.

- cmd/mini.cmd: normal batch launcher.
- cmd/build_windows.cmd: script that starts packager build flow.
- cmd/mini-shell.cmd: opens a shell for language execution.
- cmd/run.bat: starts server.py for local browser access.

powershell/

Purpose: PowerShell-native launch and packaging scripts.

- powershell/mini.ps1: PowerShell launcher.
- powershell/build_windows.ps1: PowerShell build packaging script.

server/

Purpose: browser/web infrastructure for running the language in a UI.

- server/web/index.html: UI entry page.
- server/web/styles.css: interface styling.
- server/web/app.js: browser logic and example handling.
- server/web/documentation.js: documentation support data.
- server/web/workbench.js: interactive workbench tooling.
- server/web/README.md: frontend usage notes.

build/

Purpose: generated packaging artifacts and compiled outputs.

- build/mini/...: PyInstaller or build-system output for the mini executable.

legacy/

Purpose: older or historical implementation files kept for comparison/reference.

- legacy/interpreter.py: earlier interpreter version.
- legacy/lexer.py: earlier lexer implementation.
- legacy/parser.py: earlier parser implementation.
- legacy/temp.py: scratch or temporary development file.

colab/

Purpose: notebook-style or demonstration environment for cloud execution.

- colab/colab_interpreter.py: notebook-friendly interpreter version.

Summary

This project is a compact, readable interpreter implementation built around the classic phases of lexing, parsing, and evaluation. It is intentionally educational and easy to follow, while still providing a real command-line execution path, test suite, and browser integration. The runtime is organized around tokenization, syntax trees, scope-based environment management, and function execution semantics, all of which are wrapped in simple command-line and web entry points.

End of generated documentation.